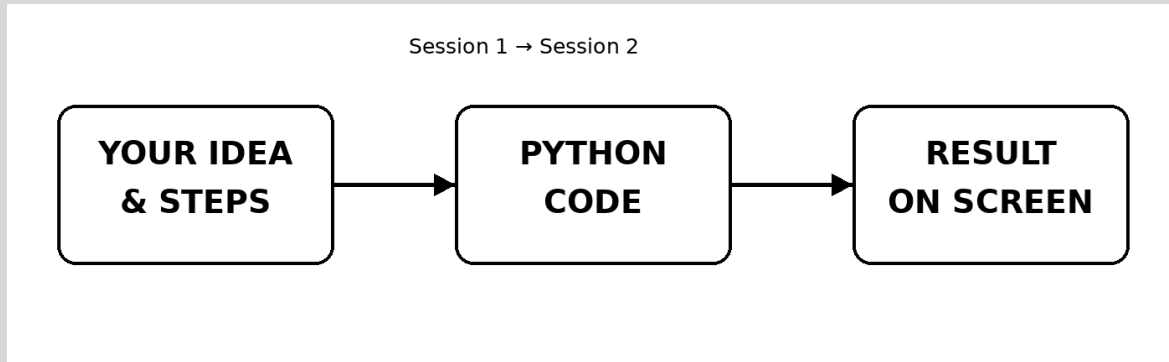


SESSION 2 – PYTHON FUNDAMENTALS

PYTHON FROM ZERO | Absolute beginner → solid foundation

In Session 1, you learned that programming starts with a problem and a clear set of steps. You even designed an algorithm for checking voting eligibility without writing Python. Today, we begin turning those ideas into Python code.



The bridge from your idea to Python code to a result.

SESSION 2 GOAL

By the end of this session, you should be able to write a small Python program that receives information, stores it, works with it, and displays a result.

1. From an Algorithm to Python

An algorithm is the plan. Python is one way of writing that plan so a computer can execute it.

A tiny example

```
name = "Ada"
print(name)
```

You do not need to memorize everything today. Focus on understanding what each line tells Python to do.

2. Python Syntax and Structure

Syntax means the rules for writing Python correctly. Think of it like spelling and sentence rules: a small mistake can change what the computer understands.

- Text is normally written inside quotes, such as "Hello".
- Parentheses are used in function calls, such as print(...).
- Python is sensitive to how code is written. Small spelling or punctuation mistakes can cause errors.
- Comments begin with # and are notes for humans.

```
# This is a comment
print("Hello")
```

3. Output: Showing Something on the Screen

The `print()` function tells Python to display something.

```
print("Hello, Python!")
print(25)
```

Try changing the words inside the quotes. Then try changing the number.

REMEMBER

What you see on the screen is the output. The instruction that produces it is your code.

4. Values and Data Types

A program works with values. Different kinds of values are useful for different jobs.

Type	Example	Used for
str	"Martin"	text
int	20	whole numbers
float	20.5	numbers with decimals
bool	True	yes/no style values

Notice the difference:

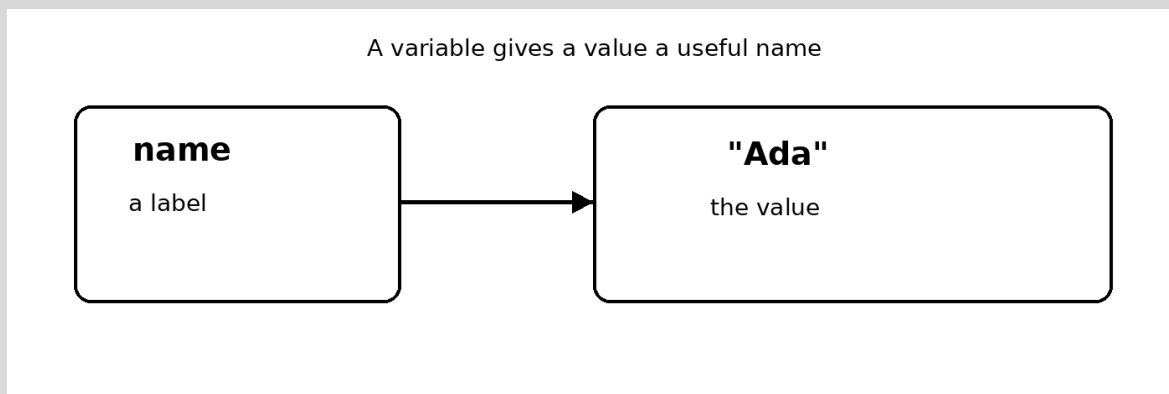
```
age = 20
age_text = "20"

print(age)
print(age_text)
```

They may look similar when printed, but Python treats them as different types.

5. Variables: Giving Values a Name

A variable is a name we give to a value so that we can use that value later. A useful analogy is a labelled container: the label helps you find what you stored.



A variable gives a value a useful name.

```
name = "Ada"
age = 20
score = 75
```

The = sign here means assignment: put the value on the right into the variable named on the left.

COMMON BEGINNER MISTAKE

Do not read `name = "Ada"` as mathematical equality. In Python, this line assigns a value to a variable.

6. Input: Letting the User Give Information

So far, we have written the information ourselves. `input()` lets the person running the program type something.

```
name = input("What is your name? ")
print("Hello", name)
```

One important detail: `input()` gives you text. If you want to do arithmetic with a number typed by the user, convert it.

```
age = input("How old are you? ")
print(type(age))
```

The result is a string (str), even when the person types 20.

```
age = int(input("How old are you? "))
print(age + 1)
```

For decimal numbers, use `float()`.

```
price = float(input("Enter the price: "))
```

7. Operators and Expressions

Operators tell Python how to work with values.

Operator	Example	Meaning
+	10 + 5	addition
-	10 - 5	subtraction
*	10 * 5	multiplication
/	10 / 5	division
%	10 % 3	remainder

Comparison operators produce a Boolean value. We will use them properly with if/else in Session 3.

```
score = 75
print(score >= 50)
print(score == 75)
```

8. Put It Together

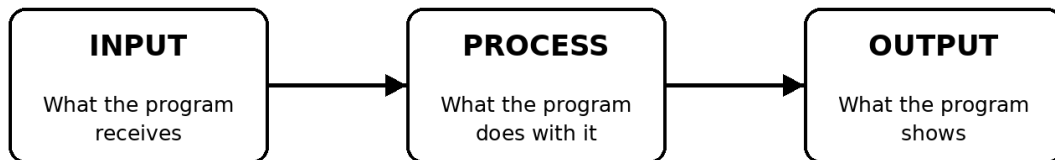
Now combine input, conversion, variables, arithmetic, and output.

```
name = input("Enter your name: ")
score1 = float(input("Enter your first score: "))
score2 = float(input("Enter your second score: "))

total = score1 + score2

print("Student:", name)
print("Total score:", total)
```

Read it from top to bottom. Ask: What information comes in? Where is it stored? What calculation happens? What comes out?



A simple way to read a small program: input → process → output.

Practice

Practice 1 — Personal Introduction

Write a program that asks for your name and age, then prints both.

```
name = input("Name: ")
age = int(input("Age: "))
print("My name is", name)
print("I am", age, "years old.")
```

Practice 2 — Simple Calculation

Ask for two numbers and print their sum.

```
first = float(input("First number: "))
second = float(input("Second number: "))
print("Total:", first + second)
```

Mini Challenge — Student Total

Ask for a student's name and three scores. Store the scores in variables and print the total. If you finish early, calculate the average too. You do not need if/else yet.

What You Should Understand Before Session 3

- How to write and run a simple Python statement.
- How `print()` displays output.
- The difference between text, whole numbers, decimals, and Boolean values.
- How variables store values under useful names.
- How `input()` receives information and why `int()` or `float()` may be needed.
- How arithmetic and comparison operators produce results.

NEXT SESSION

You already designed a voting-eligibility algorithm in Session 1. In Session 3, we will give Python the ability to follow the decision in that algorithm.

Quick Self-Check

1. What does `print()` do?
2. What is the difference between 20 and "20"?
3. What does the = sign do when assigning a variable?
4. Why would you use `int(input(...))`?
5. What kind of value does `score >= 50` produce?
6. What is the difference between an algorithm and Python code?

Assignment

Research:

1. What is an f-strings and how to use it?
2. How does if/else work?
3. What are loops in python?

SUBMIT USING THE PROVIDED GOOGLE FORM LINK OR IN THE GROUP CHAT

Martin Emmanuel
AI & Automations Expert